Jetpack Compose Basics



00 13 Topics We'll Learn

01 Introduction

02 XML vs Jetpack Compose

03 Setup

04 @Composable & Preview

05 Layouts

06 Modifiers

07 Material Design

08 State Management

09 Lists

10 Images & Resources

11 Interactions

12 Navigation

13 ViewModel

01 What Is Jetpack Compose?

Jetpack Compose is Android's modern toolkit for building UI. Instead of XML layouts, you **describe your screen with Kotlin functions** — and Compose draws it for you.

Declarative

Say *what* the UI should look like. Compose handles the updates.

Kotlin only

UI and logic live in one language. No XML files to keep in sync.

Less code

Fewer files, no adapters, and live previews in Android Studio.

It is the recommended toolkit for new Android apps, and it works side by side with your existing Views.

01 The Big Idea: UI = f(state)



Tap the button → count becomes 4 → Compose runs Counter(4) again → the UI shows “Count: 4”. This re-run is called **recomposition**.

Imperative (XML Views)

You change views by hand: find the TextView, then set its text.

Declarative (Compose)

You change the state. The UI updates itself.

02 Same Screen, Two Ways

Old way XML + Kotlin

```
<!-- res/layout/activity_main.xml -->
<LinearLayout ...>
    <TextView
        android:id="@+id/greeting"
        android:text="Hello!" />
</LinearLayout>

// MainActivity.kt
val tv: TextView =
    findViewById(R.id.greeting)
tv.text = "Hello, Ana!"
```

New way Jetpack Compose

```
// MainActivity.kt
setContent {
    Greeting("Ana")
}

@Composable
fun Greeting(name: String) {
    Text("Hello, $name!")
}

// No XML file needed!
```

02 XML vs Compose at a Glance

Topic	XML Views	Jetpack Compose
Language	XML + Kotlin/Java	Kotlin only
Style	Imperative: “how”	Declarative: “what”
Update UI	Find view, set value	Change state → auto update
Lists	RecyclerView + Adapter	LazyColumn
Preview	Layout Editor	@Preview functions
Per screen	Layout XML + Activity	One Kotlin file

They work together! Use ComposeView to put Compose inside XML, or AndroidView to put an old View inside Compose.

03 Setting Up a Compose Project

1. **New Project**
   Pick **Empty Activity**. It's Compose-ready.

2. **Gradle is ready**
   Plugin, BOM and Material 3 are already added.

3. **setContent { }**
   Show composables instead of an XML layout.

BOM = one version number that keeps all Compose libraries in sync.

```
// app/build.gradle.kts (simplified)
plugins {
    alias(libs.plugins.kotlin.compose)
}
android {
    buildFeatures { compose = true }
}
dependencies {
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.activity.compose)
    implementation(libs.androidx.material3)
    debugImplementation(libs.androidx.ui.tooling)
}
```

04 @Composable: Functions That Draw UI

- Add **@Composable** to a Kotlin function — now it can draw UI.
- Name it with a **Capital letter**, like a noun: Greeting.

```
setContent { }
```

```
Greeting(name)
```

```
Text("Hello, Ana!")
```

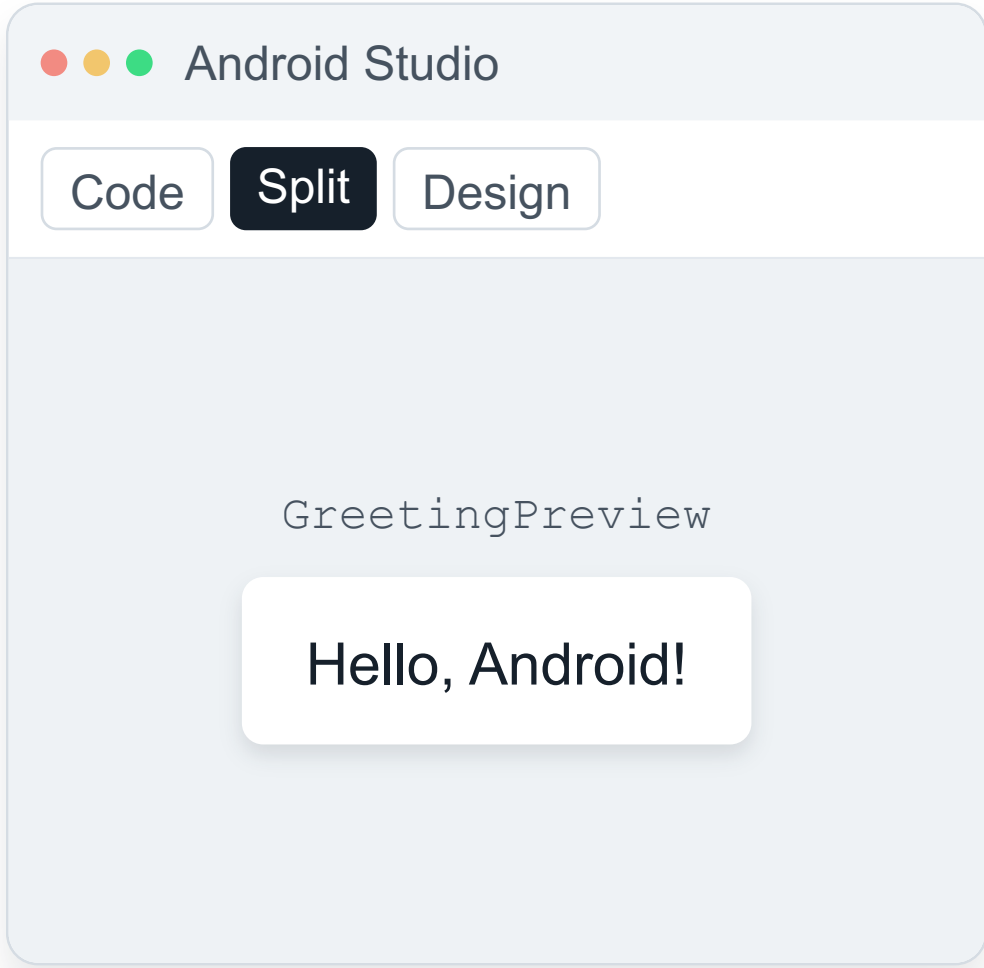
```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}

class MainActivity : ComponentActivity() {
    override fun onCreate(state: Bundle?) {
        super.onCreate(state)
        setContent {
            Greeting(name = "Ana")
        }
    }
}
```

04 @Preview: See UI Without Running

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MyAppTheme {
        Greeting(name = "Android")
    }
}
```

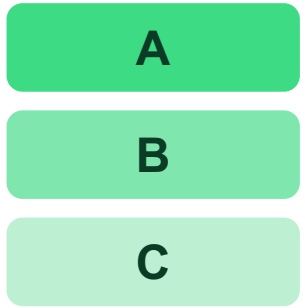
- ✓ A preview function takes **no parameters**.
- ✓ Add more @Preview tags for dark mode, big fonts or tablets.
- ✓ Use **Interactive Mode** to click buttons inside the preview.



05 Layouts: Column, Row and Box

Column

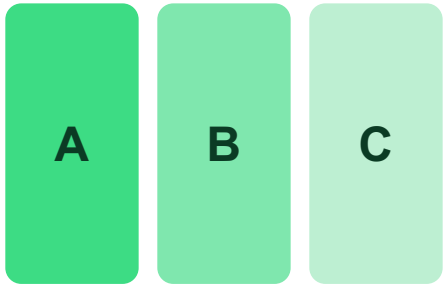
Puts items **top to bottom**




```
Column { A(); B(); C() }
```

Row

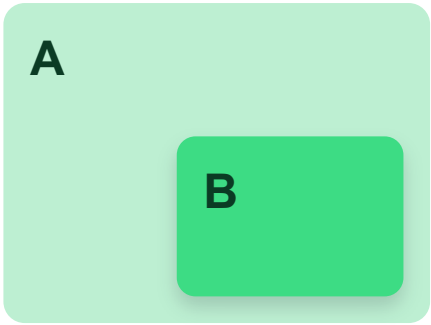
Puts items **side by side**




```
Row { A(); B(); C() }
```

Box

Stacks items **on top** of each other

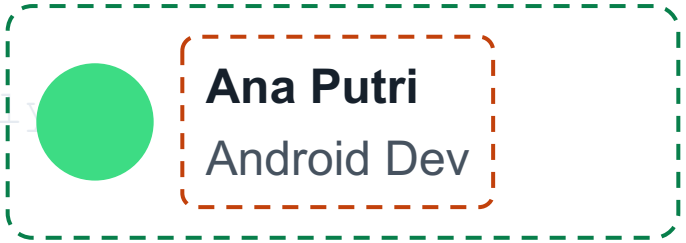



```
Box { A(); B() }
```

05 Putting Layouts Together

```
@Composable
fun ProfileCard() {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Avatar() // your own composable
        Spacer(Modifier.width(16.dp))
        Column {
            Text("Ana Putri")
            Text("Android Dev")
        }
    }
}
```

Result on screen:



Row: side by side

Column: top to bottom

Spacer: empty gap

06 Modifiers: Style and Behavior

A **Modifier** changes how a composable looks or behaves — size, spacing, color, clicks. Chain them with dots.

```
Text(
    text = "Hello Compose",
    modifier = Modifier
        .fillMaxWidth()
        .background(Color(0xFFE8F5E9))
        .padding(16.dp)
        .clickable { /* do something */ }
)
```

Modifier	What it does
.padding(16.dp)	Space around
.size(48.dp)	Fixed size
.fillMaxWidth()	Use all width
.background(…)	Paint behind
.clip(CircleShape)	Cut to a shape
.clickable { }	React to taps

06 Modifier Order Matters!

Modifiers apply in order, top to bottom — like layers of wrapping paper.

```
Modifier
    .padding(24.dp)
    .background(Color.Green)
```



Padding first → the space is **outside** the color.

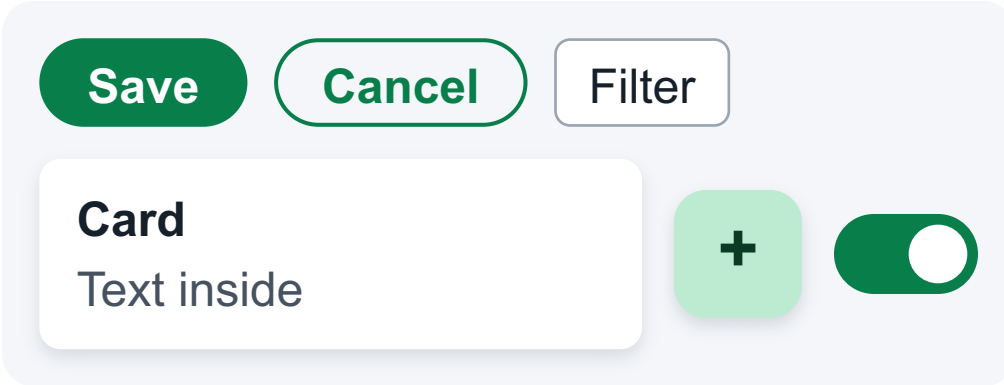
```
Modifier
    .background(Color.Green)
    .padding(24.dp)
```



Background first → the space is **inside** the color.

07 Material Design 3: A Ready-Made Look

- **Theme** — one place for your colors, fonts and shapes.
- **Components** — Button, Card, TextField, TopAppBar, Scaffold…
- **Dynamic color** — on Android 12+, colors can follow the wallpaper.




```
MyAppTheme { // made for you in ui/theme/Theme.kt
    Column {
        Button(onClick = { /* save */ }) { Text("Save") }
        Text("Hi!", color = MaterialTheme.colorScheme.primary)
    }
}
```

08 State: Data That Changes the UI

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0
    Button(onClick = { count++ }) {
        Text("Clicked $count times")
    }
}
```



Compose runs Counter() again with the new value.

`mutableStateOf(0)`

A value Compose watches. Change it → UI redraws.

`remember { }`

Keeps the value when the function runs again.

`rememberSaveable { }`

Also survives screen rotation.

08 State Hoisting: Lift State Up

`CounterScreen()`

Owns the state: **count**

`count` ↓

↑ `onAdd()`

`Counter(count, onAdd)`

Just shows it — easy to reuse and test.

State goes down · Events go up

```
@Composable
fun CounterScreen() {
    var count by remember { mutableStateOf(0) }
    Counter(count, onAdd = { count++ })
}

@Composable
fun Counter(count: Int, onAdd: () -> Unit) {
    Button(onClick = onAdd) {
        Text("Clicked $count times")
    }
}
```

09 Lists with LazyColumn

Lazy = only the items you can see are created. Even 1,000 items scroll smoothly.

```
LazyColumn {
    items(1000) { index ->
        Text(
            text = "Item #$index",
            modifier = Modifier.padding(16.dp)
        )
    }
}
```



Item #6 … #999
not created yet

LazyRow scrolls sideways. **LazyVerticalGrid** makes a grid.

10 Images and Resources

Put files in **res/**. Android gives each one an ID in the **R** class.

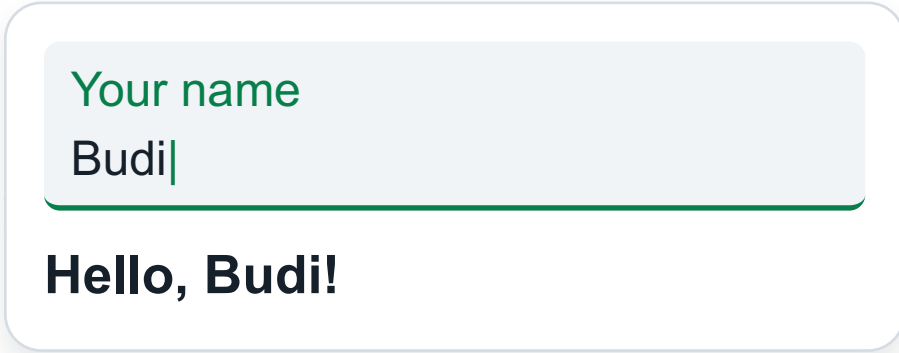
```
drawable/cat.png
→ R.drawable.cat
values/strings.xml
→ R.string.welcome
```

Image from a URL? Use the **Coil** library's AsyncImage.

```
// 1. A picture from res/drawable
Image(
    painter = painterResource(R.drawable.cat),
    contentDescription = "A cute cat",
    contentScale = ContentScale.Crop,
    modifier = Modifier.size(120.dp).clip(CircleShape)
)
// 2. Text from res/values/strings.xml
Text(stringResource(R.string.welcome))
// 3. A built-in Material icon
Icon(Icons.Filled.Star, contentDescription = null)
```

11 Interactions: Clicks and Text Input

```
@Composable
fun NameForm() {
    var name by remember { mutableStateOf("") }
    Column {
        TextField(
            value = name,
            onValueChange = { name = it },
            label = { Text("Your name") }
        )
        Text("Hello, $name!")
    }
}
```



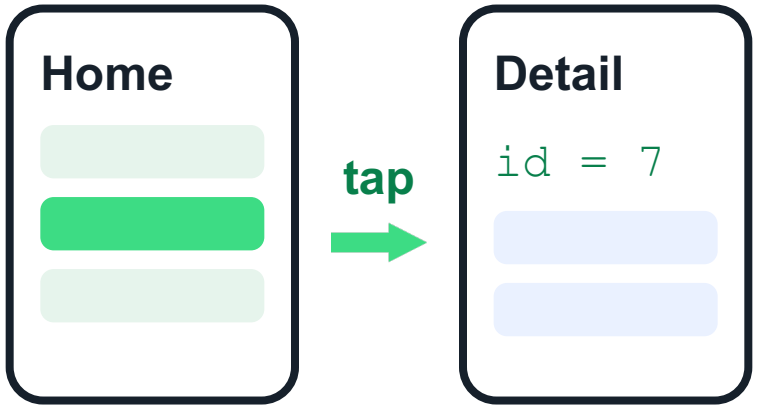
Pattern: **value** in, **onValueChange** out.

```
Button(onClick = { })
```

```
Modifier.clickable { }
```

```
Switch(checked,
onCheckedChange)
```

12 Navigation Between Screens

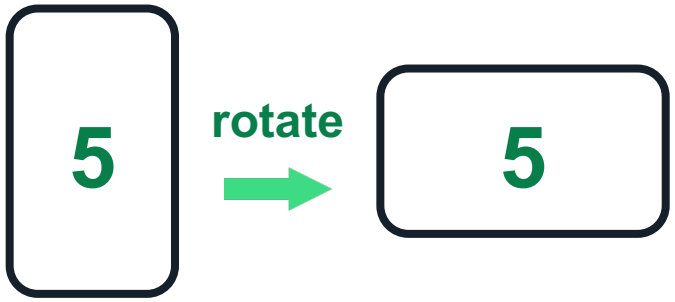


1. Routes are **@Serializable** classes.
2. **NavHost** maps each route to a screen.
3. **navigate()** to go; Back returns.

```
@Serializable object Home
@Serializable data class Detail(val id: Int)

NavHost(navController, startDestination = Home) {
    composable<Home> {
        HomeScreen(onOpen = { id ->
            navController.navigate(Detail(id))
        })
    }
    composable<Detail> { entry ->
        DetailScreen(entry.toRoute<Detail>().id)
    }
}
```

13 ViewModel: State That Survives



The count is still 5 after rotation.

remember forgets on rotation.
ViewModel keeps your data.

The lifecycle-aware collector pauses when the app is hidden.

```
class CounterViewModel : ViewModel() {
    private val _count = MutableStateFlow(0)
    val count = _count.asStateFlow()
    fun add() { _count.value++ }
}

@Composable
fun CounterScreen(vm: CounterViewModel = viewModel())
    val count by vm.count.collectAsStateWithLifecycle(
    Counter(count, onAdd = vm::add)
}
```